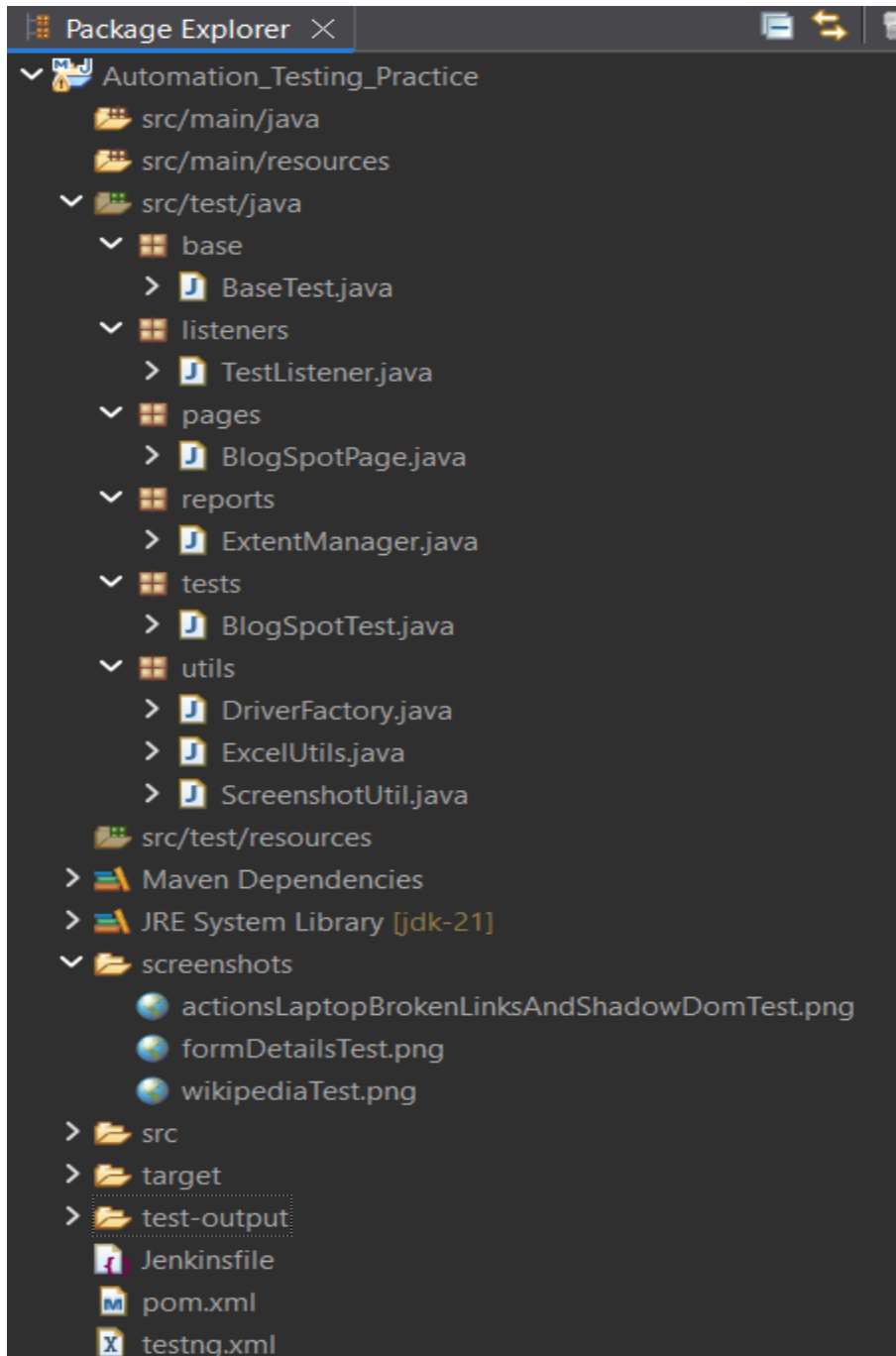


Automation Practice BlogSpot

Code:-

Project Structure:-



I. src/test/java/base/BaseTest.java:-

```
package base;
```

```
import java.time.Duration;
```

```
import org.openqa.selenium.WebDriver;
```

```
import
```

```
org.openqa.selenium.support.ui.WebDriverWait;
```

```
import org.testng.annotations.AfterClass;
```

```
import org.testng.annotations.BeforeClass;
```

```
import utils.DriverFactory;
```

```
public class BaseTest {
```

```
    protected WebDriver driver;
```

```
    public WebDriver getDriver() {
```

```
        return driver;
```

```
    }
```

```
    protected WebDriverWait wait;
```

```
    @BeforeClass
```

```
    public void setup() {
```

```
        driver = DriverFactory.initializeDriver();
```

```
        driver.manage().timeouts()
```

```
            .implicitlyWait(Duration.ofSeconds(0));
```

```
driver.get("https://testautomationpractice.blogspot  
.com/");
```

```
        wait = new WebDriverWait(driver,  
Duration.ofSeconds(15));  
    }
```

```
    @AfterClass  
    public void tearDown() {  
  
        DriverFactory.quitDriver();  
    }  
}
```

II. src/test/java/listeners/TestListener.java:-

```
package listeners;
```

```
import org.testng.ITestContext;  
import org.openqa.selenium.WebDriver;  
import base.BaseTest;  
import utils.ScreenshotUtil;  
import org.testng.ITestListener;  
import org.testng.ITestResult;
```

```
public class TestListener implements  
ITestListener {
```

```
@Override
public void onStart(ITestContext context) {

    System.out.println("===== TEST SUITE
STARTED =====");
}
```

```
@Override
public void onTestStart(ITestResult result) {

    System.out.println(
        "STARTED : "
        +
result.getMethod().getMethodName());
}
```

```
@Override
public void onTestSuccess(ITestResult result) {

    System.out.println(
        "PASSED : "
        +
result.getMethod().getMethodName());
}
```

```
@Override
public void onTestFailure(ITestResult result) {

    System.out.println(
```

```
        "FAILED : "  
        +  
result.getMethod().getMethodName());  
  
        System.out.println(  
            "REASON : "  
            + result.getThrowable());  
  
        Object currentClass = result.getInstance();  
  
        WebDriver driver =  
            ((BaseTest) currentClass).getDriver();  
  
        ScreenshotUtil.captureScreenshot(  
            driver,  
            result.getMethod().getMethodName());  
    }
```

```
    @Override  
    public void onTestSkipped(ITestResult result) {  
  
        System.out.println(  
            "SKIPPED : "  
            +  
result.getMethod().getMethodName());  
    }
```

```
    @Override  
    public void onFinish(ITestContext context) {
```

```
        System.out.println("===== TEST SUITE  
FINISHED =====");  
    }  
}
```

III. src/test/java/pages/BlogSpotPage.java:-

```
package pages;
```

```
import java.time.Duration;
```

```
import org.openqa.selenium.Alert;
```

```
import org.openqa.selenium.By;
```

```
import org.openqa.selenium.JavascriptExecutor;
```

```
import org.openqa.selenium.SearchContext;
```

```
import org.openqa.selenium.WebDriver;
```

```
import org.openqa.selenium.WebElement;
```

```
import org.openqa.selenium.interactions.Actions;
```

```
import
```

```
org.openqa.selenium.support.ui.ExpectedCondi  
tions;
```

```
import org.openqa.selenium.support.ui.Select;
```

```
import
```

```
org.openqa.selenium.support.ui.WebDriverWait;
```

```
public class BlogSpotPage {
```

```
    private WebDriver driver;
```

```
    private WebDriverWait wait;
```

```

private Actions actions;

public BlogSpotPage(WebDriver driver) {
    this.driver = driver;
    this.wait = new WebDriverWait(driver,
Duration.ofSeconds(15));
    this.actions = new Actions(driver);
}

// =====
// FORM SECTION
// =====

    public void fillUserDetails(String name, String
email, String phone, String address) {

wait.until(ExpectedConditions.visibilityOfElement
Located(By.id("name")))
        .clear();

driver.findElement(By.id("name")).sendKeys(nam
e);

wait.until(ExpectedConditions.visibilityOfElement
Located(By.id("email")))
        .clear();

```

```
driver.findElement(By.id("email")).sendKeys(email);
```

```
wait.until(ExpectedConditions.visibilityOfElementLocated(By.id("phone")))
    .clear();
```

```
driver.findElement(By.id("phone")).sendKeys(phone);
```

```
wait.until(ExpectedConditions.visibilityOfElementLocated(By.id("textarea")))
    .clear();
```

```
driver.findElement(By.id("textarea")).sendKeys(address);
}
```

```
public void selectGenderAndDays() {
```

```
wait.until(ExpectedConditions.elementToBeClickable(By.id("male")))
    .click();
```



```
wait.until(ExpectedConditions.elementToBeClicka  
ble(By.id("saturday")))  
    .click();
```

```
wait.until(ExpectedConditions.elementToBeClicka  
ble(By.id("sunday")))  
    .click();  
}
```

```
public void selectCountry() {
```

```
    WebElement country =
```

```
wait.until(ExpectedConditions.visibilityOfElement  
Located(By.id("country")));
```

```
    Select select = new Select(country);
```

```
    select.selectByIndex(9);  
}
```

```
public void selectColorsAndAnimals() {
```

```
wait.until(ExpectedConditions.elementToBeClicka  
ble(  
    By.xpath("//*[@id='colors']/option[7]")))
```

```
.click();
```

```
wait.until(ExpectedConditions.elementToBeClicka  
ble(
```

```
By.xpath("//*[@id='animals']/option[8]")))
        .click();
    }
```

```
// =====  
// DATE PICKERS  
// =====
```

```
public void enterDateOne() {
```

```
wait.until(ExpectedConditions.visibilityOfElement  
Located(By.id("datepicker")))
        .sendKeys("19/07/2002");
    }
```

```
public void selectDateTwo() {
```

```
wait.until(ExpectedConditions.elementToBeClic  
kable(By.id("txtDate")))
        .click();
```

```

WebElement month = wait.until(

ExpectedConditions.visibilityOfElementLocated(
    By.className("ui-datepicker-month")));

Select monthSelect = new Select(month);
monthSelect.selectByIndex(2);

WebElement year = wait.until(

ExpectedConditions.visibilityOfElementLocated(
    By.xpath("//select[@aria-label='Select
year']")));

Select yearSelect = new Select(year);
yearSelect.selectByVisibleText("2016");

wait.until(ExpectedConditions.elementToBeClicka
ble(
    By.xpath("//table[@class='ui-datepicker-
calendar']//a[text()='11']")))
    .click();

}

public void selectDateRange() {

```

```
wait.until(ExpectedConditions.visibilityOfElement
Located(By.id("start-date")))
        .sendKeys("03/01/2003");
```

```
wait.until(ExpectedConditions.visibilityOfElement
Located(By.id("end-date")))
        .sendKeys("05/06/2026");
```

```
wait.until(ExpectedConditions.elementToBeClicka
ble(
        By.className("submit-btn")))
        .click();
}
```

```
// =====
// FILE UPLOAD
// =====
```

```
public void uploadSingleFile(String path) {
```

```
wait.until(ExpectedConditions.presenceOfElemen
tLocated(
        By.id("singleFileInput")))
        .sendKeys(path);
```

```
wait.until(ExpectedConditions.elementToBeClickable(
```

```
By.xpath("//*[@id='singleFileForm']/button")))
        .click();
    }
```

```
public void uploadMultipleFiles(String files) {
```

```
wait.until(ExpectedConditions.presenceOfElementLocated(
    By.id("multipleFilesInput")))
        .sendKeys(files);
```

```
wait.until(ExpectedConditions.elementToBeClickable(
```

```
By.xpath("//*[@id='multipleFilesForm']/button")))
        .click();
    }
```

```
// =====
// PAGINATION TABLE
// =====
```

```
public void selectPaginationTableRows() {
```

```
wait.until(ExpectedConditions.elementToBeClicka  
ble(
```

```
By.xpath("//*[@id='productTable']/tbody/tr[1]/td[4]/i  
nput"))))  
        .click();
```

```
wait.until(ExpectedConditions.elementToBeClicka  
ble(
```

```
By.xpath("//*[@id='productTable']/tbody/tr[3]/td[4]/i  
nput"))))  
        .click();
```

```
wait.until(ExpectedConditions.elementToBeClicka  
ble(
```

```
By.xpath("//*[@id='productTable']/tbody/tr[5]/td[4]/i  
nput"))))  
        .click();  
    }
```

```
// =====  
// DYNAMIC BUTTONS  
// =====
```

```
public void handleInputFormSection() {
```

```
    wait.until(ExpectedConditions.elementToBeClickable(By.id("input1")))
        .sendKeys("Hello");
```

```
    wait.until(ExpectedConditions.elementToBeClickable(By.id("btn1")))
        .click();
```

```
    wait.until(ExpectedConditions.elementToBeClickable(By.id("input2")))
        .sendKeys("Hii");
```

```
    wait.until(ExpectedConditions.elementToBeClickable(By.id("btn2")))
        .click();
```

```
    wait.until(ExpectedConditions.elementToBeClickable(By.id("input3")))
        .sendKeys("Bye");
```

```
wait.until(ExpectedConditions.elementToBeClicka
ble(By.id("btn3")))
        .click();
    }
```

```
// =====
// WIKIPEDIA
// =====
```

```
public void searchWikipedia(String keyword) {
```

```
    wait.until(ExpectedConditions.visibilityOfElement
Located(
        By.id("Wikipedia1_wikipedia-search-
input")))
        .sendKeys(keyword);
```

```
    wait.until(ExpectedConditions.elementToBeClicka
ble(
        By.className("wikipedia-search-
button")))
        .click();
    }
```

```
public void clickStartButton() {
```



```
wait.until(ExpectedConditions.elementToBeClicka
ble(
    By.className("start")))
    .click();
}
```

```
// =====
// ALERTS
// =====
```

```
public void handleSimpleAlert() {
```

```
    WebElement alertBtn =
```

```
wait.until(ExpectedConditions.visibilityOfElement
Located(
```

```
    By.id("alertBtn")));
```

```
    ((JavascriptExecutor) driver)
        .executeScript(
```

```
"arguments[0].scrollIntoView({block:'center'});",
        alertBtn);
```

```
    alertBtn.click();
```

```
    Alert alert =
```

```
wait.until(ExpectedConditions.alertIsPresent());
```

```
        alert.accept();  
    }
```

```
    public void handleConfirmAlert() {
```

```
        driver.findElement(By.id("confirmBtn")).click();
```

```
        Alert alert =  
        wait.until(ExpectedConditions.alertIsPresent());
```

```
        alert.dismiss();  
    }
```

```
    public void handlePromptAlert() {
```

```
        driver.findElement(By.id("promptBtn")).click();
```

```
        Alert alert =  
        wait.until(ExpectedConditions.alertIsPresent());
```

```
        alert.sendKeys("Harry Potter");
```

```
        alert.accept();  
    }
```

```
    // =====
```

```
// WINDOWS & TABS
```

```
// =====
```

```
public void closeNewTab() {
```

```
    String parent = driver.getWindowHandle();
```

```
    wait.until(ExpectedConditions.elementToBeClickable(  
        By.id("PopUp")))
```

```
        .click();
```

```
        wait.until(d -> d.getWindowHandles().size()  
> 1);
```

```
        for (String handle :  
            driver.getWindowHandles()) {
```

```
            if (!handle.equals(parent)) {
```

```
                driver.switchTo().window(handle);
```

```
                driver.close();
```

```
            }
```

```
        }
```

```
        driver.switchTo().window(parent);
```

```
    }
```

```

public void closePopupWindow() {

    String parent = driver.getWindowHandle();

    wait.until(ExpectedConditions.elementToBeClicka
ble(

By.xpath("//*[@id='HTML4']/div[1]/button")))
        .click());

    wait.until(d -> d.getWindowHandles().size()
> 1);

    for (String handle :
driver.getWindowHandles()) {

        if (!handle.equals(parent)) {

            driver.switchTo().window(handle);

            driver.close();

            break;
        }
    }

    driver.switchTo().window(parent);

```

```
}
```

```
// =====
```

```
// ACTIONS
```

```
// =====
```

```
public void mouseHover() {
```

```
    WebElement menu =
```

```
wait.until(ExpectedConditions.visibilityOfElement  
Located(
```

```
        By.className("dropbtn")));
```

```
    ((JavascriptExecutor) driver)
```

```
        .executeScript(
```

```
"arguments[0].scrollIntoView({block:'center'});",  
        menu);
```

```
    actions.moveToElement(menu).perform();
```

```
}
```

```
public void doubleClickButton() {
```

```
    WebElement button =
```

```
wait.until(ExpectedConditions.elementToBeClicka  
ble(
```

```
By.xpath("//*[@id='HTML10']/div[1]/button"))));
```

```
    actions.doubleClick(button).perform();  
}
```

```
public void dragAndDrop() {
```

```
    WebElement source =
```

```
wait.until(ExpectedConditions.visibilityOfElement  
Located(
```

```
    By.id("draggable"))));
```

```
    ((JavascriptExecutor) driver)  
        .executeScript(
```

```
"arguments[0].scrollIntoView({block:'center'});",  
    source);
```

```
    WebElement target =
```

```
wait.until(ExpectedConditions.visibilityOfElement  
Located(
```

```
    By.id("droppable"))));
```

```
        actions.dragAndDrop(source,  
target).perform();  
    }
```

```
public void moveSlider() {
```

```
    WebElement slider =
```

```
wait.until(ExpectedConditions.visibilityOfElement  
Located(  
    By.xpath("//*[@id='slider-  
range']/span[1]"))));
```

```
        ((JavascriptExecutor) driver)  
        .executeScript(  
            "arguments[0].scrollIntoView({block:'center'});",  
            slider);
```

```
        ((JavascriptExecutor) driver)  
        .executeScript(  
            "arguments[0].scrollIntoView({block:'center'});",  
            slider);
```

```
        ((JavascriptExecutor) driver)  
        .executeScript(  
            "arguments[0].scrollIntoView({block:'center'});",  
            slider);
```

```
        actions.dragAndDropBy(slider, 100,  
0).perform();  
    }
```

```
public void selectScrollableDropdown() {
```

```
wait.until(ExpectedConditions.elementToBeClicka  
ble(  
    By.id("comboBox")))
```

```
.click();
```

```
WebElement option =
```

```
wait.until(ExpectedConditions.elementToBeClickable(  
ble(  

```

```
By.xpath("//*[@id='dropdown']/div[7]"))));
```

```
((JavascriptExecutor) driver)  
.executeScript(  

```

```
"arguments[0].scrollIntoView({block:'center'});",  
option);
```

```
option.click();  
}
```

```
public void handleLaptopLink() {
```

```
// =====  
// LAPTOP LINK (Apple)  
// =====
```

```
WebElement laptopLink = wait.until(  

```

```
ExpectedConditions.elementToBeClickable(  
By.linkText("Apple"))));
```



```
((JavascriptExecutor) driver)
    .executeScript(

"arguments[0].scrollIntoView({block:'center'});",
    laptopLink);

    laptopLink.click();

wait.until(ExpectedConditions.urlContains("apple"
));

    System.out.println("==== APPLE PAGE
====");
    System.out.println("Page Title : " +
driver.getTitle());
    System.out.println("Current URL : " +
driver.getCurrentUrl());

    driver.navigate().back();

wait.until(ExpectedConditions.visibilityOfElement
Located(
    By.id("name")));

    System.out.println("Returned to Home Page
after Apple link");
}
```

```

// =====
// BROKEN LINK (404)
// =====
public void handleBrokenLink() {
    WebElement broken404 = wait.until(

ExpectedConditions.elementToBeClickable(
    By.linkText("Errorcode 404")));

    ((JavascriptExecutor) driver)
        .executeScript(

"arguments[0].scrollIntoView({block:'center'});",
    broken404);

    broken404.click();

wait.until(ExpectedConditions.urlContains("404"))
;

    System.out.println("==== 404 ERROR
PAGE =====");
    System.out.println("Page Title : " +
driver.getTitle());
    System.out.println("Current URL : " +
driver.getCurrentUrl());

```

```
driver.navigate().back();
```

```
wait.until(ExpectedConditions.visibilityOfElement  
Located(  
    By.id("name")));
```

```
    System.out.println("Returned to Home Page  
after 404 link");
```

```
    }  
    // =====  
    // SHADOW DOM  
    // =====
```

```
    public void handleShadowDom(String filePath)  
{
```

```
        WebElement shadowHost =
```

```
wait.until(ExpectedConditions.visibilityOfElement  
Located(  
    By.id("shadow_host")));
```

```
        SearchContext shadowRoot =  
shadowHost.getShadowRoot();
```

```
        WebElement textBox =
```

```
shadowRoot.findElement(By.cssSelector("input[ty  
pe='text']"));
```

```
    textBox.clear();  
    textBox.sendKeys("Hii");
```

```
WebElement checkbox =
```

```
shadowRoot.findElement(By.cssSelector("input[ty  
pe='checkbox']"));
```

```
    if (!checkbox.isSelected()) {  
        checkbox.click();  
    }
```

```
WebElement upload =
```

```
shadowRoot.findElement(By.cssSelector("input[ty  
pe='file']"));
```

```
    upload.sendKeys(filePath);  
}  
// =====  
// ASSERTION METHODS  
// =====
```

```
public String getEnteredName() {
```

```
        return driver.findElement(By.id("name"))
            .getAttribute("value");
    }
```

```
public String getSelectedCountry() {
```

```
    Select select = new Select(
        driver.findElement(By.id("country")));
```

```
    return
select.getFirstSelectedOption().getText();
}
```

```
public String getDateOneValue() {
```

```
    return driver.findElement(By.id("datepicker"))
        .getAttribute("value");
}
```

```
public boolean isPaginationRowSelected() {
```

```
    return driver.findElement(
By.xpath("//*[@id='productTable']/tbody/tr[1]/td[4]/i
nput"))
        .isSelected();
}
```

```
public String getInput1Value() {
```

```
        return driver.findElement(By.id("input1"))
            .getAttribute("value");
    }
```

```
    public String getInput2Value() {
        return driver.findElement(By.id("input2"))
            .getAttribute("value");
    }
```

```
    public String getInput3Value() {
        return driver.findElement(By.id("input3"))
            .getAttribute("value");
    }
```

```
    public boolean isWikipediaResultDisplayed() {
```

```
        return wait.until(
            ExpectedConditions.visibilityOfElementLocated(
                By.xpath("//div[@id='wikipedia-
search-result-link']")))
            .isDisplayed();
    }
```

```
    public int getWindowCount() {
```

```
        return driver.getWindowHandles().size();
    }
```

```
public String getDroppedText() {  
  
    return driver.findElement(By.id("droppable"))  
        .getText();  
}
```

```
public String getShadowInputValue() {  
  
    SearchContext shadowRoot =  
        driver.findElement(By.id("shadow_host"))  
            .getShadowRoot();  
  
    return shadowRoot  
  
        .findElement(By.cssSelector("input[type='text']"))  
            .getAttribute("value");  
}
```

```
public boolean isShadowCheckboxSelected() {  
  
    SearchContext shadowRoot =  
        driver.findElement(By.id("shadow_host"))  
            .getShadowRoot();  
  
    return shadowRoot  
  
        .findElement(By.cssSelector("input[type='checkbox']"))  
            .isSelected();  
}
```

```
}
```

```
public String getCurrentUrl() {  
    return driver.getCurrentUrl();  
}
```

```
public String getPageTitle() {  
    return driver.getTitle();  
}  
}
```

IV. src/test/java/tests/BlogSpotTest.java:-

```
package tests;
```

```
import org.testng.annotations.BeforeClass;  
import org.testng.annotations.Test;
```

```
import base.BaseTest;  
import pages.BlogSpotPage;  
import utils.ExcelUtils;  
import org.testng.Assert;
```

```
public class BlogSpotTest extends BaseTest {
```

```
    private BlogSpotPage page;
```

```
    @BeforeClass
```



```
public void setupPage() {
    page = new BlogSpotPage(driver);
}

@Test(priority = 1)
public void formDetailsTest() {

    String path = "D:\\ExcelTest\\testdata.xlsx";

    Object[] data =
ExcelUtils.getSingleRowData(path, "Sheet1");

    String name = data[0].toString();
    String email = data[1].toString();
    String phone = data[2].toString();
    String address = data[3].toString();

    page.fillUserDetails(name, email, phone,
address);

    page.selectGenderAndDays();

    System.out.println("Validating Name
Field...");

    Assert.assertEquals(
        page.getEnteredName(),
        name,
        "Name field validation failed");
}
```

```
        System.out.println("Name Field Validation  
Passed");  
    }
```

```
@Test(priority = 2)  
public void dropdownSelectionTest() {
```

```
    page.selectCountry();
```

```
    page.selectColorsAndAnimals();
```

```
        System.out.println("Validating Country  
Selection...");
```

```
        Assert.assertEquals(  
            page.getSelectedCountry(),  
            "India");
```

```
        System.out.println("Country Selection  
Validation Passed");  
    }
```

```
@Test(priority = 3)  
public void datePickerTest() {
```

```
    page.enterDateOne();
```

```
        System.out.println("Validating Date Picker  
Value...");
```

```
        Assert.assertEquals(  
            page.getDateOneValue(),  
            "19/07/2002");
```

```
        System.out.println("Date Picker Validation  
Passed");
```

```
        page.selectDateTwo();
```

```
        page.selectDateRange();  
    }
```

```
@Test(priority = 4)  
public void fileUploadTest() {
```

```
    page.uploadSingleFile("E:\\Text\\ABC.txt");
```

```
    page.uploadMultipleFiles(  
        "E:\\Text2\\T\\f.txt\n"  
        + "E:\\Text2\\T\\file2.txt\n"  
        + "E:\\Text2\\T\\t.txt");
```

```
    Assert.assertTrue(true);  
}
```

```
@Test(priority = 5)
```

```
public void paginationTableTest() {

    page.selectPaginationTableRows();

    System.out.println("Validating Pagination
Table Selection...");

    Assert.assertTrue(
        page.isPaginationRowSelected(),
        "Pagination row checkbox not
selected");

    System.out.println("Pagination Table
Validation Passed");
}

@Test(priority = 6)
public void inputFormSectionTest() {

    page.handleInputFormSection();

    System.out.println("Validating Input Form
Section...");

    Assert.assertEquals(page.getInput1Value(),
"HHello");
    Assert.assertEquals(page.getInput2Value(),
"Hii");
```

```
        Assert.assertEquals(page.getInput3Value(),  
"Bye");
```

```
        System.out.println("Input Form Validation  
Passed");  
    }
```

```
@Test(priority = 7)  
public void wikipediaTest() {
```

```
    page.searchWikipedia("Java");
```

```
    System.out.println("Validating Wikipedia  
Search Result...");
```

```
    Assert.assertTrue(  
        page.isWikipediaResultDisplayed(),  
        "Wikipedia result not displayed");
```

```
    System.out.println("Wikipedia Validation  
Passed");
```

```
    page.clickStartButton();  
}
```

```
@Test(priority = 8)  
public void alertTest() {
```

```
    page.handleSimpleAlert();
```

```
        page.handleConfirmAlert();

        page.handlePromptAlert();

        Assert.assertTrue(true);
    }

    @Test(priority = 9)
    public void windowAndTabTest() {

        page.closePopupWindow();

        page.closeNewTab();

        System.out.println("Validating Window
Count...");

        Assert.assertEquals(
            page.getWindowCount(),
            1,
            "More than one window remains
open");

        System.out.println("Window Validation
Passed");
    }

    @Test(priority = 10)
```

```
public void
actionsLaptopBrokenLinksAndShadowDomTest()
{

    page.mouseHover();

    page.doubleClickButton();

    page.dragAndDrop();

    System.out.println("Validating Drag and Drop
Result...");

    Assert.assertEquals(
        page.getDroppedText(),
        "Dropped!",
        "Drag and Drop failed");

    System.out.println("Drag and Drop Validation
Passed");

    page.moveSlider();

    page.selectScrollableDropdown();

    page.handleLaptopLink();

    page.handleBrokenLink();
```

```
page.handleShadowDom("E:\\Text\\ABC.txt");
```

```
    System.out.println("Validating Shadow DOM  
Input...");
```

```
    Assert.assertEquals(  
        page.getShadowInputValue(),  
        "Hii");
```

```
    Assert.assertTrue(  
        page.isShadowCheckboxSelected());
```

```
    System.out.println("Shadow DOM Validation  
Passed");  
    }  
}
```

V. src/test/java/utils/DriverFactory.java:-

```
package utils;
```

```
import org.openqa.selenium.WebDriver;  
import  
org.openqa.selenium.chrome.ChromeDriver;
```

```
import  
io.github.bonigarcia.wdm.WebDriverManager;
```

```
public class DriverFactory {
```



```
private static WebDriver driver;

private DriverFactory() {
}

public static WebDriver initializeDriver() {

    if (driver == null) {

        WebDriverManager.chromedriver().setup();

        driver = new ChromeDriver();

        driver.manage().window().maximize();
    }

    return driver;
}

public static WebDriver getDriver() {

    return driver;
}

public static void quitDriver() {

    if (driver != null) {
```

```
        driver.quit();

        driver = null;
    }
}
}
```

VI. src/test/java/utils/ExcelUtils.java:-

```
package utils;

import java.io.FileInputStream;

import
org.apache.poi.ss.usermodel.DataFormatter;
import org.apache.poi.xssf.usermodel.*;

public class ExcelUtils {

    public static Object[] getSingleRowData(String
filePath, String sheetName) {

        Object[] data = new Object[4];

        try {
            FileInputStream fis = new
FileInputStream(filePath);
```

```

        XSSFWorkbook workbook = new
XSSFWorkbook(fis);
        XSSFSheet sheet =
workbook.getSheet(sheetName);

        XSSFRow row = sheet.getRow(1);

        DataFormatter formatter = new
DataFormatter();

        for (int i = 0; i < 4; i++) {
            data[i] =
formatter.formatCellValue(row.getCell(i));
        }

        workbook.close();
        fis.close();

    } catch (Exception e) {
        e.printStackTrace();
    }

    return data;
}
}

```

VII. src/test/java/utils/ScreenshotUtil.java:-

```

package utils;

```

```
import java.io.File;
import java.io.IOException;
import java.nio.file.Files;
import java.nio.file.StandardCopyOption;

import org.openqa.selenium.OutputType;
import org.openqa.selenium.TakesScreenshot;
import org.openqa.selenium.WebDriver;

public class ScreenshotUtil {

    public static void
captureScreenshot(WebDriver driver, String
testName) {

    try {

        File source = ((TakesScreenshot) driver)
            .getScreenshotAs(OutputType.FILE);

        File destination = new File(
            "screenshots/" + testName + ".png");

        destination.getParentFile().mkdirs();

        Files.copy(
            source.toPath(),
            destination.toPath(),
```

```
StandardCopyOption.REPLACE_EXISTING);
```

```
        System.out.println(
            "Screenshot saved: "
            +
            destination.getAbsolutePath());
```

```
    } catch (IOException e) {
```

```
        e.printStackTrace();
```

```
    }
```

```
}
```

```
}
```

VIII. src/test/java/reports/ExtentManager.java:-

```
package reports;
```

```
import
```

```
com.aventstack.extentreports.ExtentReports;
```

```
import
```

```
com.aventstack.extentreports.reporter.ExtentSparkReporter;
```

```
public class ExtentManager {
```

```
    private static ExtentReports extent;
```

```
    public static ExtentReports getExtentReport() {
```

```

    if (extent == null) {

        ExtentSparkReporter sparkReporter =
            new ExtentSparkReporter(
                "test-
output/ExtentReport.html");

        sparkReporter.config()
            .setReportName("BlogSpot
Automation Report");

        sparkReporter.config()
            .setDocumentTitle("Execution
Report");

        extent = new ExtentReports();

        extent.attachReporter(sparkReporter);
    }

    return extent;
}
}

```

IX. Jenkinsfile:-

```

pipeline {

```

agent any

```
tools {
    maven 'Maven3'
    jdk 'jdk21'
}
```

stages {

```
stage('Checkout') {
    steps {
        git branch: 'main',
            url:
'https://github.com/asrafulmd12/Wipro-traning.git'
    }
}
```

```
stage('Build') {
    steps {
        dir('Automation_Testing_Practice') {
            bat 'mvn clean'
        }
    }
}
```

```
stage('Compile') {
    steps {
        dir('Automation_Testing_Practice') {
            bat 'mvn compile'
```

```
    }  
  }  
}
```

```
stage('Test') {  
  steps {  
    dir('Automation_Testing_Practice') {  
      bat 'mvn test'  
    }  
  }  
}
```

```
stage('Package') {  
  steps {  
    dir('Automation_Testing_Practice') {  
      bat 'mvn package'  
    }  
  }  
}
```

```
}
```

```
post {
```

```
  success {  
    echo 'Build Successful'  
  }  
}
```



```

        failure {
            echo 'Build Failed'
        }
    }
}

```

X. pom.xml:-

```

<project
xmlns="http://maven.apache.org/POM/4.0.0"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-
instance"
xsi:schemaLocation="http://maven.apache.org/P
OM/4.0.0 https://maven.apache.org/xsd/maven-
4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>

    <groupId>Automation_Testing_Practice</groupId>
    <

    <artifactId>Automation_Testing_Practice</artifact
Id>
    <version>0.0.1-SNAPSHOT</version>
    <dependencies>
    <!-- Source:
https://mvnrepository.com/artifact/org.testng/testn
g -->
    <dependency>
        <groupId>org.testng</groupId>

```

```
<artifactId>testng</artifactId>
<version>7.12.0</version>
<scope>test</scope>
</dependency>
<!-- Source:
https://mvnrepository.com/artifact/org.seleniumhq.
selenium/selenium-java -->
<dependency>
  <groupId>org.seleniumhq.selenium</groupId>
  <artifactId>selenium-java</artifactId>
  <version>4.44.0</version>
  <scope>compile</scope>
</dependency>
<dependency>
  <groupId>org.apache.poi</groupId>
  <artifactId>poi-ooxml</artifactId>
  <version>5.2.5</version>
</dependency>
<dependency>
  <groupId>io.github.bonigarcia</groupId>
  <artifactId>webdrivermanager</artifactId>
  <version>6.3.2</version>
</dependency>
</dependencies>
<build>
<plugins>
<plugin>
<groupId>org.apache.maven.plugins</groupId>
<artifactId>maven-surefire-plugin</artifactId>
```

```
<version>3.5.3</version>
<configuration>
<suiteXmlFiles>
<suiteXmlFile>testng.xml</suiteXmlFile>
</suiteXmlFiles>
</configuration>
</plugin>
</plugins>
</build>
</project>
```

XI. testng.xml:-

```
<!DOCTYPE suite SYSTEM
"https://testng.org/testng-1.0.dtd">
```

```
<suite name="Suite">
```

```
    <listeners>
        <listener class-
name="listeners.TestListener"/>
    </listeners>
```

```
    <test name="BlogSpotTests">
```

```
        <classes>
            <class name="tests.BlogSpotTest"/>
        </classes>
```

```
    </test>
```

</suite>

O/P:-

===== TEST SUITE STARTED =====

STARTED : formDetailsTest

Validating Name Field...

Name Field Validation Passed

PASSED : formDetailsTest

STARTED : dropdownSelectionTest

Validating Country Selection...

Country Selection Validation Passed

PASSED : dropdownSelectionTest

STARTED : datePickerTest

Validating Date Picker Value...

Date Picker Validation Passed

PASSED : datePickerTest

STARTED : fileUploadTest

PASSED : fileUploadTest

STARTED : paginationTableTest

Validating Pagination Table Selection...

Pagination Table Validation Passed

PASSED : paginationTableTest

STARTED : inputFormSectionTest

Validating Input Form Section...

Input Form Validation Passed

PASSED : inputFormSectionTest

STARTED : wikipediaTest

Validating Wikipedia Search Result...

Wikipedia Validation Passed

PASSED : wikipediaTest

STARTED : alertTest

PASSED : alertTest

STARTED : windowAndTabTest

Validating Window Count...

Window Validation Passed

PASSED : windowAndTabTest

STARTED :

actionsLaptopBrokenLinksAndShadowDomTest

Validating Drag and Drop Result...

Drag and Drop Validation Passed

===== APPLE PAGE =====

Page Title : Apple

Current URL : <https://www.apple.com/>

Returned to Home Page after Apple link

===== 404 ERROR PAGE =====

Page Title : 404 - File or directory not found.

Current URL : <http://www.deadlinkcity.com/error-page.asp?e=404>

Returned to Home Page after 404 link

Validating Shadow DOM Input...

Shadow DOM Validation Passed

PASSED :

actionsLaptopBrokenLinksAndShadowDomTest

===== TEST SUITE FINISHED =====

=====

=====

Suite

Total tests run: 10, Passes: 10, Failures: 0, Skips: 0

=====

=====